

THE REAL-WORLD STORY

Imagine guessing a secret number between 1 and 100. Every time you guess, the host tells you either "Too High" or "Too Low".

THE BINARY SEARCH AHA MOMENT

Guessing 50 splits the search space EXACTLY in half! If too low, eliminate 1..50. Next guess 75. Search space shrinks exponentially: 100 → 50 → 25 → 12 → 6 → 3 → 1. Found in at most 7 steps ($\log_2 100$)!

WHY LINEAR SEARCH FAILS

- Linear Search $O(N)$:** Checks elements 1 by 1. For $N = 1,000,000,000$ elements, requires up to 1 Billion checks!
- Binary Search $O(\log N)$:** Needs at most **30 checks** for 1 Billion elements!
- Monotonic Property:** Works on ANY search space that transitions monotonically from 'False → True'!

SEARCH RANGE NARROWING

Safe Middle Index Formula:

```
// Prevents Integer Overflow when (L + R) > Integer.MAX_VALUE!  
int mid = left + (right - left) / 2;  
  
If arr[mid] < target → left = mid + 1  
If arr[mid] > target → right = mid - 1  
If arr[mid] == target → Found!
```

PREREQUISITES & COMPLEXITY REASONING

Pattern / Operation	Time Complexity	Auxiliary Space
Classic Binary Search (Exact Value)	$O(\log N)$	$O(1)$ in-place
Lower Bound / Upper Bound	$O(\log N)$	$O(1)$ space
Search in Rotated Sorted Array	$O(\log N)$	$O(1)$ space
Search on Answer (Koko Eating Bananas)	$O(N \log(\text{Max} - \text{Min}))$	$O(1)$ space

CLUES THAT SCREAM BINARY SEARCH

- "Array is *SORTED*" and target index requested → Classic Binary Search / Lower Bound
- "Find minimum speed / maximum capacity satisfying condition C" → Search on Answer
- "Sorted array rotated at unknown pivot" → Modified Rotated Binary Search

PATTERN
1

SEARCH IN ROTATED SORTED ARRAY (Half Sorted
Check)

e.g. Search in Rotated Sorted Array (LC
33)

WHEN TO USE

Array is rotated at pivot. Determine which half
(`left..mid` or `mid..right`) is strictly sorted, then
check if target lies within that sorted range!

TEMPLATE — LC 33

```
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l ≤ r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        // Left half is sorted:
        if (nums[l] ≤ nums[mid]) {
            if (nums[l] ≤ target && target < nums[mid]) r = mid - 1;
            else l = mid + 1;
        } else { // Right half is sorted:
            if (nums[mid] < target && target ≤ nums[r]) l = mid + 1;
            else r = mid - 1;
        }
    }
    return -1;
}
```

PATTERN 2 **SEARCH A 2D MATRIX (Virtual 1D Flattening)** e.g. Search a 2D Matrix (LC 74)

WHEN TO USE

Flatten M x N matrix virtually to 1D index `0 .. M\*N - 1`. Row = `mid / cols`, Col = `mid % cols` in O(log(M · N)) time.

TEMPLATE — LC 74

```
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int l = 0, r = rows * cols - 1;
    while (l ≤ r) {
        int mid = l + (r - l) / 2;
        int val = matrix[mid / cols][mid % cols];
        if (val == target) return true;
        if (val < target) l = mid + 1; else r = mid - 1;
    }
    return false;
}
```

PATTERN
4

FIND MINIMUM IN ROTATED SORTED ARRAY (Pivot Search)

e.g. Find Minimum in Rotated Sorted Array (LC 153)

RIGHT ELEMENT COMPARISON

Compare `nums[mid]` against `nums[right]`. If `nums[mid] > nums[right]`, pivot lies on the right half (`l = mid + 1`). Otherwise pivot is on left half (`r = mid`)!

TEMPLATE — LC 153

```
public int findMin(int[] nums) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] > nums[r]) {
            l = mid + 1; // Pivot is in right half!
        } else {
            r = mid; // Pivot is mid or in left half!
        }
    }
    return nums[l];
}
```

PATTERN
5

MEDIAN OF TWO SORTED ARRAYS (Binary Search on Partition)

e.g. Median of Two Sorted Arrays (LC 4)

PARTITION MATH $O(\log(\min(M, N)))$

Binary search partition `i` on smaller array A. Calculate corresponding partition `j` on array B. Check `A[i-1] <= B[j]` and `B[j-1] <= A[i]`!

TEMPLATE — LC 4

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B, A);
    int m = A.length, n = B.length;
    int l = 0, r = m;
    while (l <= r) {
        int i = l + (r - l) / 2;
        int j = (m + n + 1) / 2 - i;
        int maxLeftA = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
        int minRightA = (i == m) ? Integer.MAX_VALUE : A[i];
        int maxLeftB = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
        int minRightB = (j == n) ? Integer.MAX_VALUE : B[j];
        if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
            if ((m + n) % 2 == 1) return Math.max(maxLeftA, maxLeftB);
            return (Math.max(maxLeftA, maxLeftB) + Math.min(minRightA, minRightB)) / 2.0;
        } else if (maxLeftA > minRightB) r = i - 1;
        else l = i + 1;
    }
    return 0.0;
}
```

BINARY SEARCH

MASTERCLASS

Decision Tree & Trigger Words

COMPLETE BINARY SEARCH PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
704	Binary Search	Classic Binary Search	$O(\log N)$	$O(1)$	E
33	Search in Rotated Sorted Array	Rotated Half Check	$O(\log N)$	$O(1)$	M
74	Search a 2D Matrix	Virtual 1D Flattening	$O(\log(M \cdot N))$	$O(1)$	M
875	Koko Eating Bananas	Search on Answer	$O(N \log(\text{Max}))$	$O(1)$	M
153	Find Min in Rotated Array	Pivot Search	$O(\log N)$	$O(1)$	M
34	First & Last Position	Lower & Upper Bound	$O(\log N)$	$O(1)$	M
4	Median of Two Sorted Arrays	Binary Search on Partition	$O(\log(\min(M, N)))$	$O(1)$	H

LC 704 · Binary Search

EASY

Pattern: Classic BS
Key Insight: Standard `l <= r` convergence.

```
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (nums[m] == target) return m;
        if (nums[m] < target) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

LC 33 · Search in Rotated Array

MEDIUM

Pattern: Rotated Half Check
Key Insight: Check if left half is sorted.

```
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (nums[m] == target) return m;
        if (nums[l] <= nums[m]) {
            if (nums[l] <= target && target < nums[m]) r = m - 1;
            else l = m + 1;
        } else {
            if (nums[m] < target && target <= nums[r]) l = m + 1;
            else r = m - 1;
        }
    }
    return -1;
}
```

LC 74 · Search a 2D Matrix

MEDIUM

Pattern: Virtual 1D Flatten
Key Insight: `row = mid / cols`, `col = mid % cols`.

```
public boolean searchMatrix(int[][] matrix, int target) {
    int R = matrix.length, C = matrix[0].length, l = 0, r = R * C - 1;
    while (l <= r) {
        int m = l + (r - l) / 2, val = matrix[m / C][m % C];
        if (val == target) return true;
        if (val < target) l = m + 1;
        else r = m - 1;
    }
    return false;
}
```


LC 875 · Koko Eating Bananas

MEDIUM

Pattern: Search on Answer

Key Insight: BS speed `1..max(piles)`.

```
public int minEatingSpeed(int[] piles, int h) {
    int l = 1, r = 1000000000;
    while (l < r) {
        int m = l + (r - l) / 2, hrs = 0;
        for (int p : piles) hrs += (p + m - 1) / m;
        if (hrs ≤ h) r = m; else l = m + 1;
    }
    return l;
}
```

LC 153 · Find Min in Rotated Array

MEDIUM

Pattern: Pivot Search

Key Insight: Compare `nums[mid]` against `nums[r]`.

```
public int findMin(int[] nums) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int m = l + (r - l) / 2;
        if (nums[m] > nums[r]) l = m + 1; else r = m;
    }
    return nums[l];
}
```

LC 4 · Median of Two Sorted Arrays

HARD

Pattern: Partition BS

Key Insight: BS partition on smaller array.

```
public double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B, A);
    int m = A.length, n = B.length, l = 0, r = m;
    while (l ≤ r) {
        int i = l + (r - l) / 2, j = (m + n + 1) / 2 - i;
        int maxLA = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
        int minRA = (i == m) ? Integer.MAX_VALUE : A[i];
        int maxLB = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
        int minRB = (j == n) ? Integer.MAX_VALUE : B[j];
        if (maxLA ≤ minRB && maxLB ≤ minRA) {
            if ((m + n) % 2 == 1) return Math.max(maxLA, maxLB);
            return (Math.max(maxLA, maxLB) + Math.min(minRA, minRB)) / 2.0;
        } else if (maxLA > minRB) r = i - 1; else l = i + 1;
    }
    return 0.0;
}
```

LC 34 · First & Last Position

MEDIUM

Pattern: Lower & Upper Bound

Key Insight: `lowerBound(target)` and `upperBound(target) - 1`.

```
public int[] searchRange(int[] nums, int target) {
    int first = lowerBound(nums, target);
    if (first == nums.length || nums[first] ≠ target) return new int[]{-1, -1};
    int last = upperBound(nums, target) - 1;
    return new int[]{first, last};
}
```

🔍 DRY RUN — Search in Rotated Array ([4, 5, 6, 7, 0, 1, 2], target = 0)

Step	l (val)	r (val)	mid (val)	Sorted Half Check	Action
1	0 (4)	6 (2)	3 (7)	Left half [4..7] is sorted ('4 <= 7'). Target 0 NOT in [4, 7).	'l = mid + 1' (4)
2	4 (0)	6 (2)	5 (1)	Left half [0..1] is sorted ('0 <= 1'). Target 0 IS in [0, 1).	'r = mid - 1' (4)
3	4 (0)	4 (0)	4 (0)	'nums[4] == 0' → Target Match Found!	Return index 4 ✅

📐 MATHEMATICAL PROOF — LOGARITHMIC TIME COMPLEXITY $O(\log N)$

Claim: Binary Search terminates in at most $\lceil \log_2 N \rceil$ steps.

Proof: At step k , search space size is $N / 2^k$. Loop terminates when $N / 2^k = 1 \implies 2^k = N \implies k = \log_2 N$. Thus, max steps required for 1 Billion items is $\log_2(10^9) \approx 30 \rightarrow$ **Exact $O(\log N)$ Proof!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Classic Binary Search in 2m without off-by-one errors

☐ Code Lower Bound template ('first idx >= target')

☐ Code Upper Bound template ('first idx > target')

☐ Write Search in Rotated Sorted Array in $O(\log N)$

☐ Code Search a 2D Matrix with virtual 1D flattening

☐ Implement Koko Eating Bananas search on answer

☐ Code Find Minimum in Rotated Sorted Array

☐ Prove why 'mid = l + (r - l) / 2' avoids overflow

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Check Which Half is Sorted First
In Rotated Array Binary Search, ALWAYS check 'if (nums[l] <= nums[mid])' to determine which side is monotonically increasing before checking if target is bounded in that range!

Rule 2 — Search on Answer Requires Monotonicity
You can apply Search on Answer ONLY if the feasibility predicate function 'isPossible(x)' is monotonic: 'False, False, False, True, True, True'!